

category: dotnet-general

tags: [async, exception-handling, sqlite, retry-queue, silent-failure, avalonia]

severity: high

created: 2026-07-30

updated: 2026-07-30

project-origin: App-Access-V2 (iAccessDesktopv2.Avalonia)

Exception nuốt im lặng trong timer khiến trạng thái lệnh kẹt "Đang thực hiện" vĩnh viễn, requeue mỗi lần restart

Tình huống gặp phải

Sau khi sửa KPI "Lệnh chờ xử lý" để đếm đúng từ `ConcurrentQueue` (xem lesson liên quan `[[kpi-counter-transient-status-race-always-zero]]`), user báo: "vừa chờ 6 lệnh xử lý xong (KPI về 0), nhưng restart app lại vẫn còn 6 lệnh chờ".

Triệu chứng / Lỗi

- KPI/queue trong RAM cho thấy lệnh đã "biến mất" (dequeue xong, coi như xử lý xong).
- Restart app → đúng 6 lệnh cũ đó xuất hiện lại trong danh sách chờ, lặp lại vô hạn.
- Không có exception/log nào xuất hiện ở đâu cả.

Nguyên nhân gốc rễ (Root Cause)

Kiểm tra trực tiếp file SQLite (`_database.db`, bảng `LogCmd`) bằng `python3 -c "import sqlite3..."` thấy 6/8 dòng bị kẹt ở `Status=1` (DANG_THUC_HIEN) — không bao giờ chuyển sang 2 (THANH_CONG) hay 3 (LOI). Chuỗi nguyên nhân:

1. `RunCMDFromServerAsync()` dequeue lệnh khỏi `ConcurrentQueue` (RAM) **TRƯỚC** khi gọi `controller.RegisterUser(...)` — nên dù xử lý fail giữa chừng, lệnh đã biến mất khỏi queue RAM (trông như "xong" từ góc nhìn KPI/queue).
2. `KztekControllerBase.RegisterUserAsync` set DB `Status=DANG_THUC_HIEN` ở dòng **ĐẦU TIÊN**, rồi mới thực hiện giao tiếp thiết bị / xử lý — nếu bất kỳ bước nào sau đó throw exception, code không bao giờ chạy tới dòng set `THANH_CONG/LOI` ở cuối.
3. Timer cha (`StartTimerRefreshStatus`) bọc toàn bộ bằng `catch (Exception) { /* nuốt exception */ }` — parity cố ý từ nguồn, nhưng hệ quả: exception biến mất hoàn toàn, không log, không dấu vết.
4. Vì DB dùng cột persistent (`LogCmd.Status`) làm nguồn "chưa xử lý" khi app khởi động lại (`GetLogCMDNotYetByDeviceID` — `WHERE Status < 2`), dòng kẹt ở `Status=1` sẽ được nạp lại **VÔ HẠN**

mỗi lần restart, dù RAM-side đã "xử lý xong" từ lâu.

Bài học tổng quát: Khi có 2 nguồn trạng thái cho cùng 1 việc — 1 nguồn RAM/transient (queue, observable collection) và 1 nguồn bền/persistent (DB row, file) — chỉ set "processing" ở nguồn bền mà không có try/catch bọc kín để ĐẢM BẢO luôn set lại "done"/"failed" (kể cả khi exception) sẽ tạo ra trạng thái ma kẹt mãi. Càng nguy hiểm hơn khi có `catch (Exception) { }` rỗng ở tầng ngoài — im lặng che mất mọi dấu hiệu của lỗi.

Giải pháp

1. Bọc try/catch quanh TOÀN BỘ phần xử lý sau khi dequeue (không chỉ phần trong device controller):

```
try
{
    var result = await controller.RegisterUser(data, AppState.Customers);
    // ... xử lý kết quả, set row.Status = "Xong"/"Lỗi" ...
}
catch (Exception ex)
{
    SystemUtils.logger.SaveSystemLog(SystemLog.CreateApplicationProcess(
        $"[RunCMDFromServerAsync] Lỗi xử lý lệnh CmdId={data.CmdId}", ex,
        EmSystemActionType.ERROR));
    TblLogCmd.UpdateStatusByCmdID(data.CmdId, (int)EmCmdStatus.LOI, ex.Message);
    await Dispatcher.UIThread.InvokeAsync(() => row.Status = "Lỗi");
}
```

2. Dọn dữ liệu kẹt hiện có trực tiếp trong DB (không có exception message gốc để biết root cause thật, chỉ có thể đánh dấu LOI thủ công để dừng requeue vô hạn):

```
import sqlite3
con = sqlite3.connect('.../_database.db')
cur = con.cursor()
cur.execute('UPDATE LogCmd SET Status=3, Detail=? WHERE Status=1', ('...',))
con.commit()
```

Áp dụng lại (How to reuse)

- Khi thấy hành vi "xử lý xong trong phiên hiện tại nhưng lặp lại y hệt sau restart" → nghi ngay có 2 nguồn trạng thái (RAM vs DB/file) không đồng bộ, và có khả năng 1 exception bị nuốt ở đâu đó giữa lúc set "processing" và set "done".
- Kiểm tra trực tiếp file DB (sqlite3/python) THAY VÌ chỉ đọc code — trạng thái thực tế trong dữ liệu là bằng chứng khách quan nhất, nhanh hơn suy luận từ code.
- Grep `catch (Exception)` RỖNG hoặc chỉ có comment "nuốt exception"/"swallow" quanh code có thao tác 2-pha (set processing → ... → set done) — đây luôn là điểm rủi ro kẹt trạng thái ma.
- Khi thêm try/catch quanh method 2-pha, PHẢI đảm bảo catch cũng ghi lại nguồn bền (DB/file) sang trạng thái kết thúc (failed), không chỉ log — nếu không, bug vẫn còn (chỉ là có log thêm).

Chú ý / Cạm bẫy (Gotchas)

- ⚠ Sau khi thêm try/catch, dữ liệu CŨ đã kẹt từ trước KHÔNG tự sửa — phải dọn thủ công 1 lần

(không có exception gốc để biết lý do lệnh cũ fail, chấp nhận đánh dấu LOI với detail chung).

- `△ catch (Exception) { }` rỗng đôi khi là "parity cố ý" giữ nguyên hành vi từ code nguồn khi migrate — cần phân biệt: giữ nguyên hành vi bên NGOÀI (không crash app) khác với việc chấp nhận mất log/mất khả năng debug. Có thể giữ "không crash" nhưng vẫn thêm log ở tầng trong sâu hơn.

Tham chiếu

- Lesson liên quan: `[[kpi-counter-transient-status-race-always-zero]]`
- Project liên quan: App-Access-V2 (MainViewModel.cs — RunCMDFromServerAsync, KztekControllerBase.cs — RegisterUserAsync, TblLogCmd.cs)